

Efficient Lazy Sequences with C++23

`std::generator`

Simplifying Coroutines for Modern C++ Ranges

8 min read · Mar 29, 2026



Sagar

Following ▾



Listen



Share

... More

C++23 · `std::generator`

Efficient lazy sequences with `std::generator`

Simplifying coroutines for modern C++ ranges. No `promise_type`. No boilerplate. Just `co_yield`.

lazy evaluation

zero-copy yield

0(1) recursion

`std::ranges`

`gcc 14 / clang 18`

- `std::generator<int> iota(int n)`
- `co_yield std::ranges::elements_of(sub)`
- `std::generator<std::string&&, std::string>`
- `std::pmr::polymorphic_allocator<>`

lazy_sequences.cpp

```
#include <generator>
#include <ranges>

std::generator<int> naturals() {
    int i = 0;
    while (true) {
        co_yield i++; // suspends here
    }
}

auto result = naturals()
    | views::transform (n*n)
    | views::filter(n%2==0)
    | views::take (10);

// Output: 0 4 16 36 64 100 144 196 256 324
// lazy end-to-end – zero intermediate allocations
```

`co_yield` - stores pointer (zero-copy) → suspends

`elements_of` - symmetric transfer ~ 0(1) depth

Allocator - pmr / arena - zero heap allocs

input_range only single-pass · move-only

If you've read my previous article — [C++20 Coroutines Demystified: Build a Generator From Scratch](#) — you've already gone through the hard part. We built a generator by hand, defined a `promise_type`, managed a `coroutine_handle`, and watched the compiler spin up that intimidating coroutine machinery behind the scenes. It was

powerful... but also a lot of work just to get something as simple as `co_yield` working.

Now, in C++23, the standard library finally catches up with the mental model we built: `std::generator<T>`. No custom promise types. No boilerplate. No hand-rolled state machines. Just write a coroutine, `co_yield` values, and let the library handle the rest. In this article, we'll take everything we learned earlier—and map it directly onto `std::generator`, uncovering how it works, what it hides, and where the sharp edges still are.

Here is the simplest possible example:

```
#include <generator>
#include <print>

std::generator<int> iota(int start, int end) {
    for (int i = start; i < end; ++i)
        co_yield i;
}

int main() {
    for (int v : iota(0, 5))
        std::print("{} ", v);
    // Output: 0 1 2 3 4
}
```

That's it. That's the hello world. Now let's rip the hood off.

Details on the Full signature

Declared in `<generator>` header like below:

```
template<
    class Ref,
    class V = void,
    class Allocator = void >
```

```
class generator
: public ranges::view_interface<generator<Ref, V, Allocator>>
```

This signature looks intimidating, but 95% of the time you'll just use `std::generator<T>`. The rest exists for control freaks (like us).

Template Parameter	Position	Default	What It Controls	Common Use
<code>Ref</code>	1st	<i>(required)</i>	The type produced by <code>co_yield</code> and the dereferenced iterator	<code>std::generator<int></code> — most people stop here
<code>Ref</code> as reference	1st	—	Yields references instead of copies	<code>std::generator<const std::string&></code> — no copy on yield
<code>Ref</code> as rvalue ref	1st	—	Yields rvalue references	<code>std::generator<std::string&&></code> — move semantics on yield
<code>V</code>	2nd	<code>void</code> (deduced from <code>Ref</code>)	What <code>range_value_t</code> reports; the "logical" element type	Rarely set explicitly — deduction from <code>Ref = const int&</code> correctly gives <code>V = int</code>
<code>V</code> explicit	2nd	—	Override deduced value type when deduction isn't what you want	Genuinely rare; only when <code>Ref</code> deduction produces the wrong <code>range_value_t</code>
<code>Allocator</code>	3rd	<code>void</code> (uses <code>::operator new</code>)	How the coroutine frame is heap-allocated	<code>std::pmr::polymorphic_allocator<></code> for arena/pool allocation

Custom allocator example:

```
std::generator<int, void, std::pmr::polymorphic_allocator<>>
my_gen(std::allocator_arg_t, std::pmr::polymorphic_allocator<> alloc) {
    co_yield 42;
}
```

The allocator must be the first parameter (after `std::allocator_arg_t`). The promise type picks it up via a special overload of `operator new`.

The `co_yield` Mechanics — What Actually Happens ?

When you write `co_yield value;`, the compiler translates it to:

```
co_await promise.yield_value(value);
```

For `std::generator`, `yield_value` does something subtle. It does not copy your value. It stores a pointer to it.

```
co_yield my_string;
```



```
promise.yield_value(my_string)
  → stores a pointer: value_ptr = std::addressof(my_string)
  → returns suspend_always{}
  → coroutine suspends
  → caller dereferences iterator → dereferences that pointer
```

This means the value is alive because the coroutine frame is alive and suspended. The local variable still exists on the coroutine frame. Brilliant and zero-copy.

But also... dangerous if you don't understand it. More on that later.

Recursive Generators with `co_yield ranges::elements_of`

This is where `std::generator` gets *really* interesting. You can yield an entire sub-generator.

```
std::generator<int> leaf() {
    co_yield 1;
```

```

        co_yield 2;
    }

    std::generator<int> tree() {
        co_yield 0;
        co_yield std::ranges::elements_of(leaf()); // delegates!
        co_yield 3;
    }

```

Output: 0 1 2 3

Why not just use a loop?

You *could* do this:

```

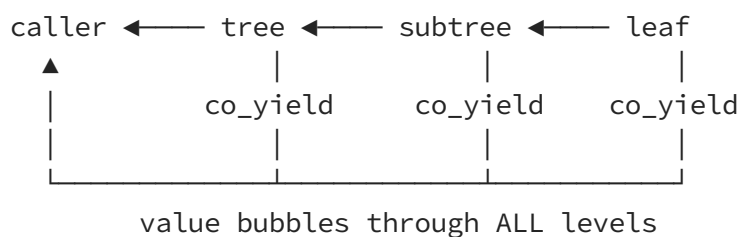
std::generator<int> tree_naive() {
    co_yield 0;
    for (auto v : leaf())
        co_yield v; // works, but...
    co_yield 3;
}

```

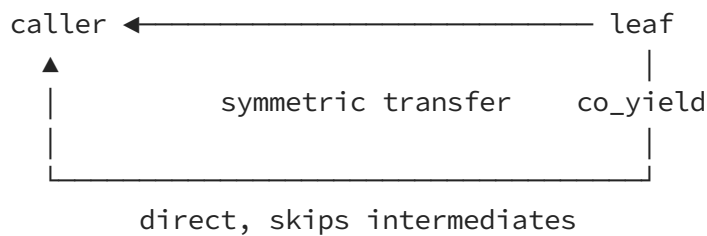
This creates a nested coroutine call stack. Every `co_yield` in `leaf()` suspends `leaf`, returns to `tree_naive`, which then `co_yield`s to the caller. If you nest 10 levels deep (like traversing a tree), every single element has to bubble through 10 suspend/resume pairs.

`elements_of` uses a different mechanism called symmetric transfer — the inner generator directly transfers control to the caller, bypassing intermediate frames.

With manual loop (O(depth) per element):



With `elements_of` (O(1) per element):



This is $O(1)$ per element instead of $O(\text{depth})$. For tree traversals, this is massive.

Hidden Tricks of `std::generator<T>`

1. HALO — The Coroutine That Never Allocates:

HALO stands for Heap Allocation eLision Optimization. If the compiler can prove that:

- The coroutine's lifetime is bounded by the caller
- The frame size is known at compile time

...it can inline the coroutine frame into the caller's stack frame, eliminating the heap allocation entirely.

```
// HALO-friendly: generator's lifetime is clearly scoped
void process() {
    for (int v : iota(0, 100)) { // generator lives only in this loop
        do_stuff(v);
    }
    // generator destroyed here, compiler can see this
}

// HALO-unfriendly: generator escapes
std::generator<int> make_gen(); // defined in another TU

void process() {
    auto g = make_gen(); // compiler can't see inside make_gen
    for (int v : g)
        do_stuff(v);
}
```

Note:

As of early 2026, most compilers (GCC, Clang, MSVC) are *terrible* at actually applying HALO. It's technically allowed by the standard, but very few real-world cases actually get elided. Don't count on it. Profile.

2. You Can `co_yield` Lvalues and Get Zero-Copy Semantics:

Because `yield_value` stores a pointer, not a copy:

```
std::generator<const std::string&> names() {
    std::string name = "Alice";
    co_yield name;    // no copy! pointer to 'name' on coroutine frame

    name = "Bob";
    co_yield name;    // still no copy, same variable, different value
}
```

But here's the trap — if you do this:

```
std::generator<const std::string&> dangerous() {
    co_yield std::string("temporary");
    // The temporary is destroyed after yield_value returns...
    // but the coroutine suspends inside yield_value...
    // so actually, is it alive?
}
```

Actually, this is fine! The temporary's lifetime is extended because `co_yield expr` is essentially `co_await promise.yield_value(expr)` — the temporary lives until the suspension point, and the coroutine suspends *inside* `yield_value`. The pointer remains valid while the coroutine is suspended.

But if you *store* the reference on the caller side and then advance the iterator, you're holding a dangling reference. Classic mistake.



3. `std::generator` Is a `std::ranges::input_range` — and ONLY That:

It's not forward, not bidirectional, not random access. Input range only. Single pass.

This means:

```
auto g = iota(0, 5);
auto it1 = g.begin();
auto it2 = g.begin(); // UNDEFINED BEHAVIOR. Don't call begin() twice.
```

You cannot:

- Sort a generator
- Use it with algorithms requiring forward iterators
- Iterate it twice

You *can*:

- Pipe it into `std::ranges::to<std::vector>()`
- Use it with `views::take`, `views::transform`, etc.
- Use it in a single range-for loop

4. The Generator Is Move-Only:

```
auto g = iota(0, 5);
auto g2 = g;           // ERROR: deleted copy constructor
auto g3 = std::move(g); // OK, but now g is empty
```

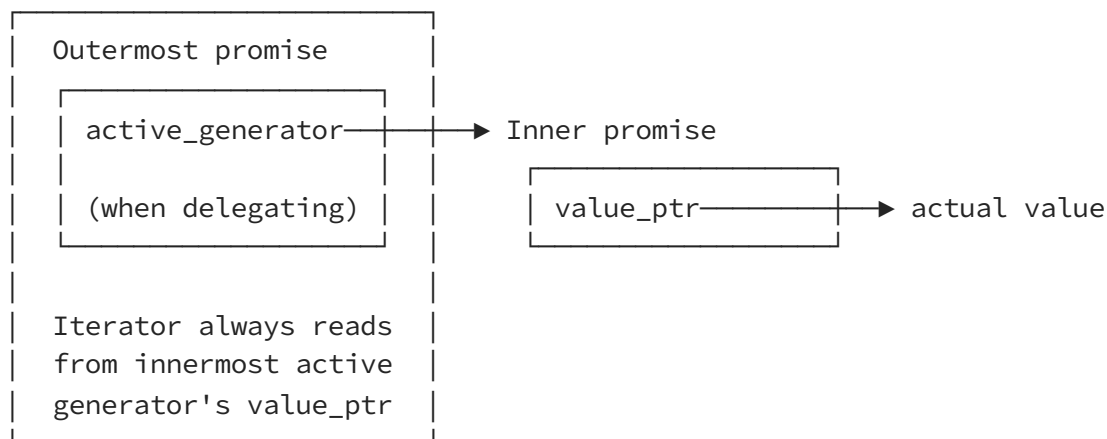

This makes sense — there's exactly one coroutine frame, and exactly one owner.

5. Yielding From a Nested Generator Preserves the Reference Category:

This is subtle. When you do `co_yield ranges::elements_of(inner_gen)`, the yielded references from the inner generator are forwarded as-is. If the inner generator yields `std::string&&`, the outer generator also yields `std::string&&` through the delegation.

This is because `elements_of` uses a special `yield_value` overload in the promise that sets up a "stack" of active generators and directly accesses the innermost one's current value.

Generator Stack (internal):



6. Exception Propagation Works Naturally:

If the coroutine body throws, the exception propagates to the caller at the point where they resume the coroutine (i.e., `++iterator` or `begin()`).

```
std::generator<int> explosive() {
    co_yield 1;
    co_yield 2;
    throw std::runtime_error("boom");
    co_yield 3; // never reached
}

int main() {
    try {
        for (int v : explosive())
```

```

        std::print("{} ", v);
    } catch (const std::exception& e) {
        std::print("caught: {}", e.what());
    }
    // Output: 1 2 caught: boom
}

```

The promise's `unhandled_exception()` rethrows via `std::rethrow_exception`. Clean and intuitive.

7. You Can Build Infinite Sequences (Obviously, But Here's the Non-Obvious Part):

```

std::generator<int> fibonacci() {
    int a = 0, b = 1;
    while (true) {
        co_yield a;
        auto next = a + b;
        a = b;
        b = next;
    }
}

```

The non-obvious part: the coroutine frame stays alive as long as the generator object exists. If you store this generator in a long-lived object, that coroutine frame (with all its locals) sits on the heap forever. There's no "timeout" or "GC". You are the garbage collector.

```

struct DataStream {
    std::generator<int> gen; // coroutine frame lives as long as this object

    DataStream() : gen(fibonacci()) {}
};
// If DataStream lives for the lifetime of your program,
// so does the coroutine frame. It's a few bytes here,
// but be aware in embedded/resource-constrained contexts.

```

8. The Allocator Hack for Arena/Pool Allocation:

In high-performance scenarios, you want to avoid `operator new` for coroutine

frames. The promise type in `std::generator` supports a special operator `new` that can intercept the allocation:

```
#include <generator>
#include <memory_resource>

// Using PMR with a monotonic buffer
char buffer[4096];
std::pmr::monotonic_buffer_resource pool{buffer, sizeof(buffer)};
std::pmr::polymorphic_allocator<> alloc{&pool};

std::generator<int, void, std::pmr::polymorphic_allocator<>>
fast_gen(std::allocator_arg_t, std::pmr::polymorphic_allocator<> a) {
    // Coroutine frame is allocated from 'a', which points to our stack buffer!
    for (int i = 0; i < 100; ++i)
        co_yield i;
}

int main() {
    for (int v : fast_gen(std::allocator_arg, alloc))
        std::print("{} ", v);
}
```

You just put a coroutine frame on a stack-backed arena. Zero heap allocations. This is the real-world alternative to HALO that actually works *today*.

9. `std::generator` Supports Reference and Value Yields Simultaneously:

This is the weirdest one. Because of how `Ref` and `V` interact:

```
// Ref = std::string&&, V = std::string
// iterator::operator* returns std::string&& (rvalue ref)
// range_value_t reports std::string
std::generator<std::string&&, std::string> move_strings() {
    std::string s = "hello";
    co_yield std::move(s); // caller gets an rvalue reference
}

int main() {
    for (auto&& s : move_strings()) {
        // s is std::string&&
        // you can move from it!
        std::string local = std::move(s);
    }
}
```

```
        std::print("{}\n", local);
    }
}
```

This enables move-out semantics from a generator. The coroutine produces values, and the caller can *steal* them. This is incredibly useful for pipelines where you're generating expensive objects and want zero-copy transfer.

Composing With Ranges

`std::generator` returns a `std::ranges::input_range`, so it plays nicely with the ranges library:

```
#include <generator>
#include <ranges>
#include <print>

std::generator<int> naturals() {
    int i = 0;
    while (true)
        co_yield i++;
}

int main() {
    // First 10 even squares
    auto result = naturals()
        | std::views::transform([](int n) { return n * n; })
        | std::views::filter([](int n) { return n % 2 == 0; })
        | std::views::take(10);

    for (int v : result)
        std::print("{} ", v);
    // Output: 0 4 16 36 64 100 144 196 256 324
}
```

This is lazy end-to-end. No intermediate allocations. The generator yields, the views transform/filter on the fly, and `take` stops after 10 elements.

Closing Thoughts

It's honestly one of the best additions in C++23. The coroutine machinery has been in the language since C++20, but `std::generator` is the first time it's *usable* without writing 200 lines of promise-type boilerplate.

`std::generator` isn't about syntax — it's about finally making coroutines usable without writing a small framework first.

Key takeaways from this post:

- `std::generator<T>` gives you Python-style `yield` in C++23
- It's lazy, single-pass, move-only, and input-range only
- Uses `co_yield` to produce values, stores pointers (zero-copy)
- `co_yield std::ranges::elements_of(...)` enables $O(1)$ recursive delegation via symmetric transfer
- Coroutine frames go on the heap by default — use custom allocators or hope for HALO
- Don't call `begin()` twice, don't hold references across `++iterator`, don't assume it's free
- Composes beautifully with `std::ranges`

Once you start thinking in lazy pipelines, it's hard to go back.

Software Development

Cpp

Functional Programming

Programming

C Plus Plus Language



Follow

Published in Towards Dev